

In the last statement a garbage value is returned to the calling function since we are not returning any specific value. Note that in this case the parentheses after **return** are dropped.

- (h) If we want that a called function should not return any value, in that case, we must mention so by using the keyword **void** as shown below.

```
void display( )
{
    printf ( "\nHeads I win..." );
    printf ( "\nTails you lose" );
}
```

- (i) A function can return only one value at a time. Thus, the following statements are invalid.

```
return ( a, b );
return ( x, 12 );
```

There is a way to get around this limitation, which would be discussed later in this chapter when we learn pointers.

- (j) If the value of a formal argument is changed in the called function, the corresponding change does not take place in the calling function. For example,

```
main( )
{
    int a = 30 ;
    fun ( a ) ;
    printf ( "\n%d", a ) ;
}

fun ( int b )
{
    b = 60 ;
```

```
    printf ( "\n%d", b );  
}
```

The output of the above program would be:

```
60  
30
```

Thus, even though the value of **b** is changed in **fun()**, the value of **a** in **main()** remains unchanged. This means that when values are passed to a called function the values present in actual arguments are not physically moved to the formal arguments; just a photocopy of values in actual argument is made into formal arguments.

Scope Rule of Functions

Look at the following program

```
main()  
{  
    int i = 20 ;  
    display ( i );  
}  
  
display ( int j )  
{  
    int k = 35 ;  
    printf ( "\n%d", j );  
    printf ( "\n%d", k );  
}
```

In this program is it necessary to pass the value of the variable **i** to the function **display()**? Will it not become automatically available to the function **display()**? No. Because by default the scope of a variable is local to the function in which it is defined. The presence

of **i** is known only to the function **main()** and not to any other function. Similarly, the variable **k** is local to the function **display()** and hence it is not available to **main()**. That is why to make the value of **i** available to **display()** we have to explicitly pass it to **display()**. Likewise, if we want **k** to be available to **main()** we will have to return it to **main()** using the **return** statement. In general we can say that the scope of a variable is local to the function in which it is defined.

Calling Convention

Calling convention indicates the order in which arguments are passed to a function when a function call is encountered. There are two possibilities here:

- (a) Arguments might be passed from left to right.
- (b) Arguments might be passed from right to left.

C language follows the second order.

Consider the following function call:

```
fun (a, b, c, d) ;
```

In this call it doesn't matter whether the arguments are passed from left to right or from right to left. However, in some function call the order of passing arguments becomes an important consideration. For example:

```
int a = 1 ;  
printf ( "%d %d %d", a, ++a, ++a ) ;
```

It appears that this **printf()** would output 1 2 3.

This however is not the case. Surprisingly, it outputs 3 3 1. This is because C's calling convention is from right to left. That is, firstly

1 is passed through the expression **a++** and then **a** is incremented to 2. Then result of **++a** is passed. That is, **a** is incremented to 3 and then passed. Finally, latest value of **a**, i.e. 3, is passed. Thus in right to left order 1, 3, 3 get passed. Once **printf()** collects them it prints them in the order in which we have asked it to get them printed (and not the order in which they were passed). Thus 3 3 1 gets printed.

One Dicey Issue

Consider the following function calls:

```
#include <conio.h>
clrscr ();
gotoxy ( 10, 20 );
ch = getch ( a );
```

Here we are calling three standard library functions. Whenever we call the library functions we must write their prototype before making the call. This helps the compiler in checking whether the values being passed and returned are as per the prototype declaration. But since we don't define the library functions (we merely call them) we may not know the prototypes of library functions. Hence when the library of functions is provided a set of '.h' files is also provided. These files contain the prototypes of library functions. But why multiple files? Because the library functions are divided into different groups and one file is provided for each group. For example, prototypes of all input/output functions are provided in the file 'stdio.h', prototypes of all mathematical functions are provided in the file 'math.h', etc.

On compilation of the above code the compiler reports all errors due to the mismatch between parameters in function call and their corresponding prototypes declared in the file 'conio.h'. You can even open this file and look at the prototypes. They would appear as shown below:

```
void clrscr( ) ;  
void gotoxy ( int, int ) ;  
int getch( ) ;
```

Now consider the following function calls:

```
#include <stdio.h>  
int i = 10, j = 20 ;  
  
printf ( "%d %d %d ", i, j ) ;  
printf ( "%d", i, j ) ;
```

The above functions get successfully compiled even though there is a mismatch in the format specifiers and the variables in the list. This is because **printf()** accepts *variable* number of arguments (sometimes 2 arguments, sometimes 3 arguments, etc.), and even with the mismatch above the call still matches with the prototype of **printf()** present in 'stdio.h'. At run-time when the first **printf()** is executed, since there is no variable matching with the last specifier **%d**, a garbage integer gets printed. Similarly, in the second **printf()** since the format specifier for **j** has not been mentioned its value does not get printed.

Advanced Features of Functions

With a sound basis of the preliminaries of C functions, let us now get into their intricacies. Following advanced topics would be considered here.

- (a) Function Declaration and Prototypes
- (b) Calling functions by value or by reference
- (c) Recursion

Let us understand these features one by one.

Function Declaration and Prototypes

Any C function by default returns an **int** value. More specifically, whenever a call is made to a function, the compiler assumes that this function would return a value of the type **int**. If we desire that a function should return a value other than an **int**, then it is necessary to explicitly mention so in the calling function as well as in the called function. Suppose we want to find out square of a number using a function. This is how this simple program would look like:

```
main()  
{  
    float a, b ;  
  
    printf ( "\nEnter any number " ) ;  
    scanf ( "%f", &a ) ;  
  
    b = square ( a ) ;  
    printf ( "\nSquare of %f is %f", a, b ) ;  
}  
  
square ( float x )  
{  
    float y ;  
  
    y = x * x ;  
    return ( y ) ;  
}
```

And here are three sample runs of this program...

```
Enter any number 3  
Square of 3 is 9.000000  
Enter any number 1.5  
Square of 1.5 is 2.000000  
Enter any number 2.5  
Square of 2.5 is 6.000000
```

The first of these answers is correct. But square of 1.5 is definitely not 2. Neither is 6 a square of 2.5. This happened because any C function, by default, always returns an integer value. Therefore, even though the function **square()** calculates the square of 1.5 as 2.25, the problem crops up when this 2.25 is to be returned to **main()**. **square()** is not capable of returning a **float** value. How do we overcome this? The following program segment illustrates how to make **square()** capable of returning a **float** value.

```
main()
{
    float square ( float );
    float a, b ;

    printf ( "\nEnter any number " );
    scanf ( "%f", &a );

    b = square ( a );
    printf ( "\nSquare of %f is %f", a, b );
}

float square ( float x )
{
    float y ;
    y = x * x ;
    return ( y );
}
```

And here is the output...

```
Enter any number 1.5
Square of 1.5 is 2.250000
Enter any number 2.5
Square of 2.5 is 6.250000
```

Now the expected answers i.e. 2.25 and 6.25 are obtained. Note that the function **square()** must be declared in **main()** as

```
float square ( float );
```

This statement is often called the prototype declaration of the **square()** function. What it means is **square()** is a function that receives a **float** and returns a **float**. We have done the prototype declaration in **main()** because we have called it from **main()**. There is a possibility that we may call **square()** from several other functions other than **main()**. Does this mean that we would need prototype declaration of **square()** in all these functions. No, in such a case we would make only one declaration outside all the functions at the beginning of the program.

In practice you may seldom be required to return a value other than an **int**, but just in case you are required to, employ the above method. In some programming situations we want that a called function should not return any value. This is made possible by using the keyword **void**. This is illustrated in the following program.

```
main()  
{  
    void gospel( );  
    gospel( );  
}  
  
void gospel( )  
{  
    printf ( "\nViruses are electronic bandits..." );  
    printf ( "\nwho eat nuggets of information..." );  
    printf ( "\nand chunks of bytes..." );  
    printf ( "\nwhen you least expect..." );  
}
```


Here, the **gospel()** function has been defined to return **void**; means it would return nothing. Therefore, it would just flash the four messages about viruses and return the control back to the **main()** function.

Call by Value and Call by Reference

By now we are well familiar with how to call functions. But, if you observe carefully, whenever we called a function and passed something to it we have always passed the ‘values’ of variables to the called function. Such function calls are called ‘calls by value’. By this what we mean is, on calling a function we are passing values of variables to it. The examples of call by value are shown below:

```
sum = calsum ( a, b, c );  
f = factr ( a );
```

We have also learnt that variables are stored somewhere in memory. So instead of passing the value of a variable, can we not pass the location number (also called address) of the variable to a function? If we were able to do so it would become a ‘call by reference’. What purpose a ‘call by reference’ serves we would find out a little later. First we must equip ourselves with knowledge of how to make a ‘call by reference’. This feature of C functions needs at least an elementary knowledge of a concept called ‘pointers’. So let us first acquire the basics of pointers after which we would take up this topic once again.

An Introduction to Pointers

Which feature of C do beginners find most difficult to understand? The answer is easy: pointers. Other languages have pointers but few use them so frequently as C does. And why not? It is C’s clever use of pointers that makes it the excellent language it is.

The difficulty beginners have with pointers has much to do with C's pointer terminology than the actual concept. For instance, when a C programmer says that a certain variable is a "pointer", what does that mean? It is hard to see how a variable can point to something, or in a certain direction.

It is hard to get a grip on pointers just by listening to programmer's jargon. In our discussion of C pointers, therefore, we will try to avoid this difficulty by explaining pointers in terms of programming concepts we already understand. The first thing we want to do is explain the rationale of C's pointer notation.

Pointer Notation

Consider the declaration,

```
int i = 3 ;
```

This declaration tells the C compiler to:

- (a) Reserve space in memory to hold the integer value.
- (b) Associate the name **i** with this memory location.
- (c) Store the value 3 at this location.

We may represent **i**'s location in memory by the following memory map.

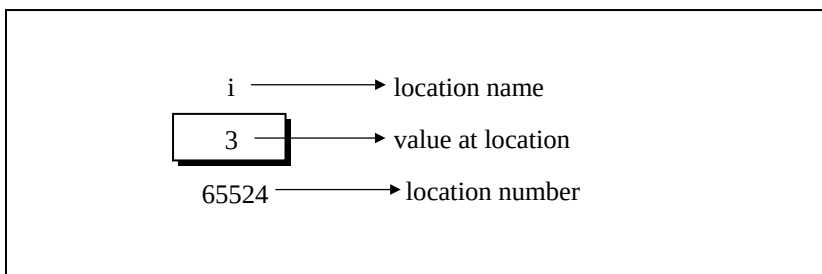


Figure 5.1

We see that the computer has selected memory location 65524 as the place to store the value 3. The location number 65524 is not a number to be relied upon, because some other time the computer may choose a different location for storing the value 3. The important point is, **i**'s address in memory is a number.

We can print this address number through the following program:

```
main()
{
    int i = 3 ;
    printf ( "\nAddress of i = %u", &i ) ;
    printf ( "\nValue of i = %d", i ) ;
}
```

The output of the above program would be:

```
Address of i = 65524
Value of i = 3
```

Look at the first **printf()** statement carefully. '**&**' used in this statement is C's 'address of' operator. The expression **&i** returns the address of the variable **i**, which in this case happens to be 65524. Since 65524 represents an address, there is no question of a sign being associated with it. Hence it is printed out using **%u**, which is a format specifier for printing an unsigned integer. We have been using the '**&**' operator all the time in the **scanf()** statement.

The other pointer operator available in C is '*****', called 'value at address' operator. It gives the value stored at a particular address. The 'value at address' operator is also called 'indirection' operator.

Observe carefully the output of the following program:

```
main()  
{  
    int i = 3 ;  
  
    printf ( "\nAddress of i = %u", &i ) ;  
    printf ( "\nValue of i = %d", i ) ;  
    printf ( "\nValue of i = %d", *( &i ) ) ;  
}
```

The output of the above program would be:

```
Address of i = 65524  
Value of i = 3  
Value of i = 3
```

Note that printing the value of ***(&i)** is same as printing the value of **i**.

The expression **&i** gives the address of the variable **i**. This address can be collected in a variable, by saying,

```
j = &i ;
```

But remember that **j** is not an ordinary variable like any other integer variable. It is a variable that contains the address of other variable (**i** in this case). Since **j** is a variable the compiler must provide it space in the memory. Once again, the following memory map would illustrate the contents of **i** and **j**.

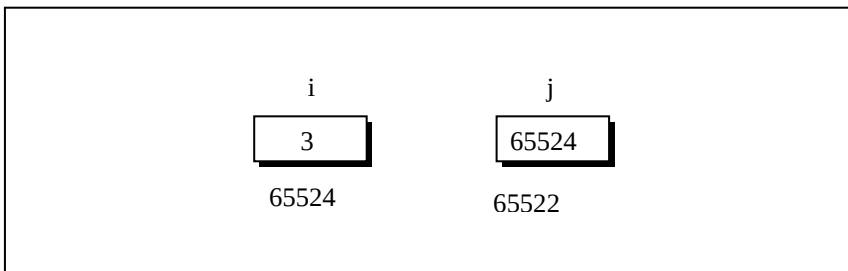


Figure 5.2

As you can see, **i**'s value is 3 and **j**'s value is **i**'s address.

But wait, we can't use **j** in a program without declaring it. And since **j** is a variable that contains the address of **i**, it is declared as,

```
int *j;
```

This declaration tells the compiler that **j** will be used to store the address of an integer value. In other words **j** points to an integer. How do we justify the usage of ***** in the declaration,

```
int *j;
```

Let us go by the meaning of *****. It stands for 'value at address'. Thus, **int *j** would mean, the value at the address contained in **j** is an **int**.

Here is a program that demonstrates the relationships we have been discussing.

```
main()
{
    int i = 3 ;
    int *j ;

    j = &i ;
    printf ( "\nAddress of i = %u", &i ) ;
    printf ( "\nAddress of i = %u", j ) ;
    printf ( "\nAddress of j = %u", &j ) ;
    printf ( "\nValue of j = %u", j ) ;
    printf ( "\nValue of i = %d", i ) ;
    printf ( "\nValue of i = %d", *( &i ) ) ;
    printf ( "\nValue of i = %d", *j ) ;
}
```

The output of the above program would be:

Address of i = 65524
Address of i = 65524
Address of j = 65522
Value of j = 65524
Value of i = 3
Value of i = 3
Value of i = 3

Work through the above program carefully, taking help of the memory locations of **i** and **j** shown earlier. This program summarizes everything that we have discussed so far. If you don't understand the program's output, or the meanings of **&i**, **&j**, ***j** and ***(&i)**, re-read the last few pages. Everything we say about C pointers from here onwards will depend on your understanding these expressions thoroughly.

Look at the following declarations,

```
int *alpha ;  
char *ch ;  
float *s ;
```

Here, **alpha**, **ch** and **s** are declared as pointer variables, i.e. variables capable of holding addresses. Remember that, addresses (location nos.) are always going to be whole numbers, therefore pointers always contain whole numbers. Now we can put these two facts together and say—pointers are variables that contain addresses, and since addresses are always whole numbers, pointers would always contain whole numbers.

The declaration **float *s** does not mean that **s** is going to contain a floating-point value. What it means is, **s** is going to contain the address of a floating-point value. Similarly, **char *ch** means that **ch** is going to contain the address of a char value. Or in other words, the value at address stored in **ch** is going to be a **char**.

The concept of pointers can be further extended. Pointer, we know is a variable that contains address of another variable. Now this variable itself might be another pointer. Thus, we now have a pointer that contains another pointer's address. The following example should make this point clear.

```
main()
{
    int i = 3, *j, **k ;

    j = &i ;
    k = &j ;
    printf ( "\nAddress of i = %u", &i ) ;
    printf ( "\nAddress of i = %u", j ) ;
    printf ( "\nAddress of i = %u", *k ) ;
    printf ( "\nAddress of j = %u", &j ) ;
    printf ( "\nAddress of j = %u", k ) ;
    printf ( "\nAddress of k = %u", &k ) ;
    printf ( "\nValue of j = %u", j ) ;
    printf ( "\nValue of k = %u", k ) ;
    printf ( "\nValue of i = %d", i ) ;
    printf ( "\nValue of i = %d", * ( &i ) ) ;
    printf ( "\nValue of i = %d", *j ) ;
    printf ( "\nValue of i = %d", **k ) ;
}
```

The output of the above program would be:

Address of i = 65524

Address of i = 65524

Address of i = 65524

Address of j = 65522

Address of j = 65522

Address of k = 65520

Value of j = 65524

Value of k = 65522

Value of i = 3
Value of i = 3
Value of i = 3
Value of i = 3

Figure 5.3 would help you in tracing out how the program prints the above output.

Remember that when you run this program the addresses that get printed might turn out to be something different than the ones shown in the figure. However, with these addresses too the relationship between **i**, **j** and **k** can be easily established.

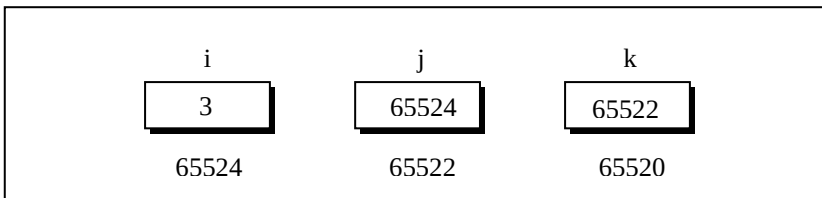


Figure 5.3

Observe how the variables **j** and **k** have been declared,

```
int i, *j, **k ;
```

Here, **i** is an ordinary **int**, **j** is a pointer to an **int** (often called an integer pointer), whereas **k** is a pointer to an integer pointer. We can extend the above program still further by creating a pointer to a pointer to an integer pointer. In principle, you would agree that likewise there could exist a pointer to a pointer to a pointer to a pointer to a pointer. There is no limit on how far can we go on extending this definition. Possibly, till the point we can comprehend it. And that point of comprehension is usually a pointer to a pointer. Beyond this one rarely requires to extend the definition of a pointer. But just in case...

Back to Function Calls

Having had the first tryst with pointers let us now get back to what we had originally set out to learn—the two types of function calls—call by value and call by reference. Arguments can generally be passed to functions in one of the two ways:

- (a) sending the values of the arguments
- (b) sending the addresses of the arguments

In the first method the ‘value’ of each of the actual arguments in the calling function is copied into corresponding formal arguments of the called function. With this method the changes made to the formal arguments in the called function have no effect on the values of actual arguments in the calling function. The following program illustrates the ‘Call by Value’.

```
main()  
{  
    int a = 10, b = 20 ;  
  
    swapv ( a, b ) ;  
    printf ( "\na = %d b = %d", a, b ) ;  
}  
  
swapv ( int x, int y )  
{  
    int t ;  
  
    t = x ;  
    x = y ;  
    y = t ;  
  
    printf ( "\nx = %d y = %d", x, y ) ;  
}
```

The output of the above program would be:

```
x = 20 y = 10  
a = 10 b = 20
```

Note that values of **a** and **b** remain unchanged even after exchanging the values of **x** and **y**.

In the second method (call by reference) the addresses of actual arguments in the calling function are copied into formal arguments of the called function. This means that using these addresses we would have an access to the actual arguments and hence we would be able to manipulate them. The following program illustrates this fact.

```
main()  
{  
    int a = 10, b = 20 ;  
  
    swapr ( &a, &b ) ;  
    printf ( "\na = %d b = %d", a, b ) ;  
}  
  
swapr( int *x, int *y )  
{  
    int t ;  
  
    t = *x ;  
    *x = *y ;  
    *y = t ;  
}
```

The output of the above program would be:

```
a = 20 b = 10
```

Note that this program manages to exchange the values of **a** and **b** using their addresses stored in **x** and **y**.

Usually in C programming we make a call by value. This means that in general you cannot alter the actual arguments. But if desired, it can always be achieved through a call by reference.

Using a call by reference intelligently we can make a function return more than one value at a time, which is not possible ordinarily. This is shown in the program given below.

```
main()
{
    int radius ;
    float area, perimeter ;

    printf ( "\nEnter radius of a circle " ) ;
    scanf ( "%d", &radius ) ;
    areaperi ( radius, &area, &perimeter ) ;

    printf ( "Area = %f", area ) ;
    printf ( "\nPerimeter = %f", perimeter ) ;
}

areaperi ( int r, float *a, float *p )
{
    *a = 3.14 * r * r ;
    *p = 2 * 3.14 * r ;
}
```

And here is the output...

```
Enter radius of a circle 5
Area = 78.500000
Perimeter = 31.400000
```

Here, we are making a mixed call, in the sense, we are passing the value of **radius** but, addresses of **area** and **perimeter**. And since we are passing the addresses, any change that we make in values stored at addresses contained in the variables **a** and **p**, would make

the change effective in **main()**. That is why when the control returns from the function **areaperi()** we are able to output the values of **area** and **perimeter**.

Thus, we have been able to indirectly return two values from a called function, and hence, have overcome the limitation of the **return** statement, which can return only one value from a function at a time.

Conclusions

From the programs that we discussed here we can draw the following conclusions:

- (a) If we want that the value of an actual argument should not get changed in the function being called, pass the actual argument by value.
- (b) If we want that the value of an actual argument should get changed in the function being called, pass the actual argument by reference.
- (c) If a function is to be made to return more than one value at a time then return these values indirectly by using a call by reference.

Recursion

In C, it is possible for the functions to call themselves. A function is called 'recursive' if a statement within the body of a function calls the same function. Sometimes called 'circular definition', recursion is thus the process of defining something in terms of itself.

Let us now see a simple example of recursion. Suppose we want to calculate the factorial value of an integer. As we know, the

factorial of a number is the product of all the integers between 1 and that number. For example, 4 factorial is $4 * 3 * 2 * 1$. This can also be expressed as $4! = 4 * 3!$ where '!' stands for factorial. Thus factorial of a number can be expressed in the form of itself. Hence this can be programmed using recursion. However, before we try to write a recursive function for calculating factorial let us take a look at the non-recursive function for calculating the factorial value of an integer.

```
main()
{
    int a, fact ;

    printf ( "\nEnter any number " );
    scanf ( "%d", &a );

    fact = factorial ( a );
    printf ( "Factorial value = %d", fact );
}

factorial ( int x )
{
    int f = 1, i ;

    for ( i = x ; i >= 1 ; i-- )
        f = f * i ;

    return ( f );
}
```

And here is the output...

```
Enter any number 3
Factorial value = 6
```

Work through the above program carefully, till you understand the logic of the program properly. Recursive factorial function can be understood only if you are thorough with the above logic.

Following is the recursive version of the function to calculate the factorial value.

```
main()  
{  
    int a, fact ;  
  
    printf ( "\nEnter any number " ) ;  
    scanf ( "%d", &a ) ;  
  
    fact = rec ( a ) ;  
    printf ( "Factorial value = %d", fact ) ;  
}  
  
rec ( int x )  
{  
    int f ;  
  
    if ( x == 1 )  
        return ( 1 ) ;  
    else  
        f = x * rec ( x - 1 ) ;  
  
    return ( f ) ;  
}
```

And here is the output for four runs of the program

```
Enter any number 1  
Factorial value = 1  
Enter any number 2  
Factorial value = 2  
Enter any number 3
```

```
Factorial value = 6
Enter any number 5
Factorial value = 120
```

Let us understand this recursive factorial function thoroughly. In the first run when the number entered through **scanf()** is 1, let us see what action does **rec()** take. The value of **a** (i.e. 1) is copied into **x**. Since **x** turns out to be 1 the condition **if (x == 1)** is satisfied and hence 1 (which indeed is the value of 1 factorial) is returned through the **return** statement.

When the number entered through **scanf()** is 2, the **(x == 1)** test fails, so we reach the statement,

```
f = x * rec ( x - 1 );
```

And here is where we meet recursion. How do we handle the expression **x * rec (x - 1)**? We multiply **x** by **rec (x - 1)**. Since the current value of **x** is 2, it is same as saying that we must calculate the value (2 * **rec (1)**). We know that the value returned by **rec (1)** is 1, so the expression reduces to (2 * 1), or simply 2. Thus the statement,

```
x * rec ( x - 1 );
```

evaluates to 2, which is stored in the variable **f**, and is returned to **main()**, where it is duly printed as

```
Factorial value = 2
```

Now perhaps you can see what would happen if the value of **a** is 3, 4, 5 and so on.

In case the value of **a** is 5, **main()** would call **rec()** with 5 as its actual argument, and **rec()** will send back the computed value. But before sending the computed value, **rec()** calls **rec()** and waits for a value to be returned. It is possible for the **rec()** that has just been

called to call yet another **rec()**, the argument **x** being decreased in value by 1 for each of these recursive calls. We speak of this series of calls to **rec()** as being different invocations of **rec()**. These successive invocations of the same function are possible because the C compiler keeps track of which invocation calls which. These recursive invocations end finally when the last invocation gets an argument value of 1, which the preceding invocation of **rec()** now uses to calculate its own **f** value and so on up the ladder. So we might say what happens is,

```
rec ( 5 ) returns ( 5 times rec ( 4 ),  
    which returns ( 4 times rec ( 3 ),  
        which returns ( 3 times rec ( 2 ),  
            which returns ( 2 times rec ( 1 ),  
                which returns ( 1 ) ) ) ) )
```

Foxed? Well, that is recursion for you in its simplest garbs. I hope you agree that it's difficult to visualize how the control flows from one function call to another. Possibly Figure 5.4 would make things a bit clearer.

Assume that the number entered through **scanf()** is 3. Using Figure 5.4 let's visualize what exactly happens when the recursive function **rec()** gets called. Go through the figure carefully. The first time when **rec()** is called from **main()**, **x** collects 3. From here, since **x** is not equal to 1, the **if** block is skipped and **rec()** is called again with the argument (**x - 1**), i.e. 2. This is a recursive call. Since **x** is still not equal to 1, **rec()** is called yet another time, with argument (2 - 1). This time as **x** is 1, control goes back to previous **rec()** with the value 1, and **f** is evaluated as 2.

Similarly, each **rec()** evaluates its **f** from the returned value, and finally 6 is returned to **main()**. The sequence would be grasped better by following the arrows shown in Figure 5.4. Let it be clear that while executing the program there do not exist so many copies of the function **rec()**. These have been shown in the figure just to

help you keep track of how the control flows during successive recursive calls.

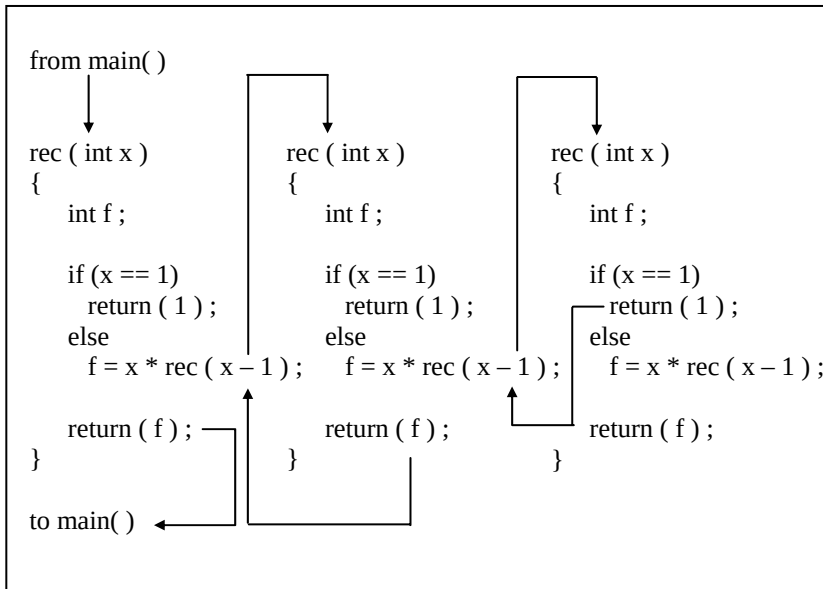


Figure 5.4

Recursion may seem strange and complicated at first glance, but it is often the most direct way to code an algorithm, and once you are familiar with recursion, the clearest way of doing so.

Recursion and Stack

There are different ways in which data can be organized. For example, if you are to store five numbers then we can store them in five different variables, an array, a linked list, a binary tree, etc. All these different ways of organizing the data are known as data structures. The compiler uses one such data structure called stack for implementing normal as well as recursive function calls.

A stack is a Last In First Out (LIFO) data structure. This means that the last item to get stored on the stack (often called Push operation) is the first one to get out of it (often called as Pop operation). You can compare this to the stack of plates in a cafeteria—the last plate that goes on the stack is the first one to get out of it. Now let us see how the stack works in case of the following program.

```
main()  
{  
    int a = 5, b = 2, c ;  
    c = add ( a, b ) ;  
    printf ( "sum = %d", c ) ;  
}  
add ( int i, int j )  
{  
    int sum ;  
    sum = i + j ;  
    return sum ;  
}
```

In this program before transferring the execution control to the function **fun()** the values of parameters **a** and **b** are pushed onto the stack. Following this the address of the statement **printf()** is pushed on the stack and the control is transferred to **fun()**. It is necessary to push this address on the stack. In **fun()** the values of **a** and **b** that were pushed on the stack are referred as **i** and **j**. In **fun()** the local variable **sum** gets pushed on the stack. When value of **sum** is returned **sum** is popped up from the stack. Next the address of the statement where the control should be returned is popped up from the stack. Using this address the control returns to the **printf()** statement in **main()**. Before execution of **printf()** begins the two integers that were earlier pushed on the stack are now popped off.

How the values are being pushed and popped even though we didn't write any code to do so? Simple—the compiler on

encountering the function call would generate code to push parameters and the address. Similarly, it would generate code to clear the stack when the control returns back from **fun()**. Figure 5.5 shows the contents of the stack at different stages of execution.

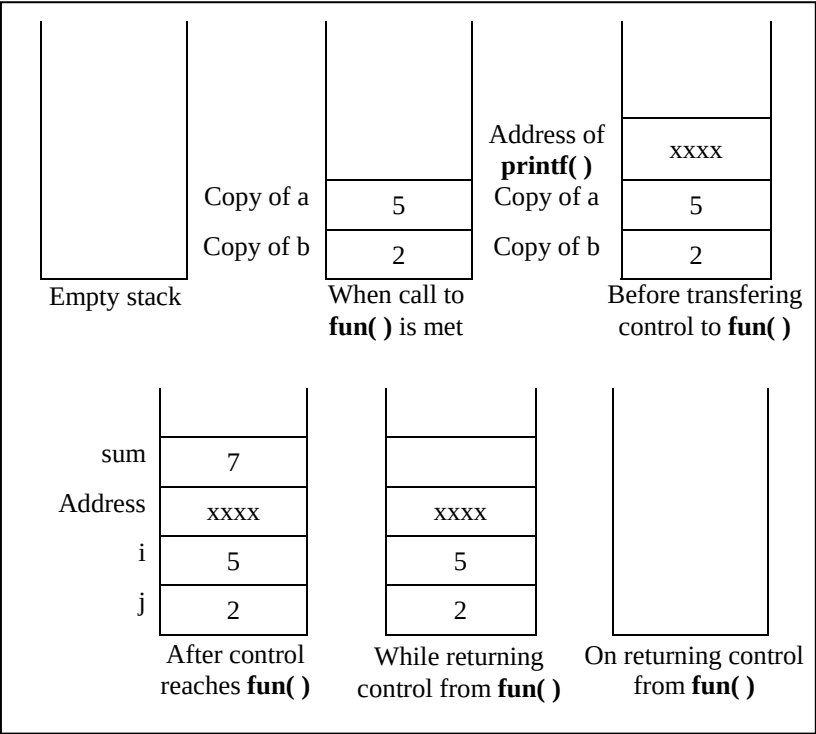


Figure 5.5

Note that in this program popping of **sum** and address is done by **fun()**, whereas popping of the two integers is done by **main()**. When it is done this way it is known as ‘CDecl Calling Convention’. There are other calling conventions as well where instead of **main()**, **fun()** itself clears the two integers. The calling convention also decides whether the parameters being passed to the function are pushed on the stack in left-to-right or right-to-left order. The standard calling convention always uses the right-to-left

order. Thus during the call to **fun()** firstly value of **b** is pushed to the stack, followed by the value of **a**.

The recursive calls are no different. Whenever we make a recursive call the parameters and the return address gets pushed on the stack. The stack gets unwound when the control returns from the called function. Thus during every recursive function call we are working with a fresh set of parameters.

Also, note that while writing recursive functions you must have an **if** statement somewhere in the recursive function to force the function to return without recursive call being executed. If you don't do this and you call the function, you will fall in an indefinite loop, and the stack will keep on getting filled with parameters and the return address each time there is a call. Soon the stack would become full and you would get a run-time error indicating that the stack has become full. This is a very common error while writing recursive functions. My advice is to use **printf()** statement liberally during the development of recursive function, so that you can watch what is going on and can abort execution if you see that you have made a mistake.

Adding Functions to the Library

Most of the times we either use the functions present in the standard library or we define our own functions and use them. Can we not add our functions to the standard library? And would it make any sense in doing so? We can add user-defined functions to the library. It makes sense in doing so as the functions that are to be added to the library are first compiled and then added. When we use these functions (by calling them) we save on their compilation time as they are available in the library in the compiled form.

Let us now see how to add user-defined functions to the library. Different compilers provide different utilities to add/delete/modify functions in the standard library. For example, Turbo C/C++

compilers provide a utility called 'tlib.exe' (Turbo Librarian). Let us use this utility to add a function **factorial()** to the library.

Given below are the steps to do so:

- (a) Write the function definition of **factorial()** in some file, say 'fact.c'.

```
int factorial ( int num )
{
    int i, f = 1 ;
    for ( i = 1 ; i <= num ; i++ )
        f = f * i ;
    return ( f ) ;
}
```

- (b) Compile the 'fact.c' file using Alt F9. A new file called 'fact.obj' would get created containing the compiled code in machine language.

- (c) Add the function to the library by issuing the command

```
C:\>tlib math.lib + c:\fact.obj
```

Here, 'math.lib' is a library filename, + is a switch, which means we want to add new function to library and 'c:\fact.obj' is the path of the '.obj' file.

- (d) Declare the prototype of the **factorial()** function in the header file, say 'fact.h'. This file should be included while calling the function.
- (e) To use the function present inside the library, create a program as shown below:

```
#include "c:\\fact.h"
main( )
```

```
{
    int f ;
    f = factorial ( 5 ) ;
    printf ( "%d", f ) ;
}
```

- (f) Compile and execute the program using Ctrl F9.

If we wish we can delete the existing functions present in the library using the minus (-) switch.

Instead of modifying the existing libraries we can create our own library. Let's see how to do this. Let us assume that we wish to create a library containing the functions **factorial()**, **prime()** and **fibonacci()**. As their names suggest, **factorial()** calculates and returns the factorial value of the integer passed to it, **prime()** reports whether the number passed to it is a prime number or not and **fibonacci()** prints the first **n** terms of the Fibonacci series, where **n** is the number passed to it. Here are the steps that need to be carried out to create this library. Note that these steps are specific to Turbo C/C++ compiler and would vary for other compilers.

- (a) Define the functions **factorial()**, **prime()** and **fibonacci()** in a file, say 'myfuncs.c'. Do not define **main()** in this file.
- (b) Create a file 'myfuncs.h' and declare the prototypes of **factorial()**, **prime()** and **fibonacci()** in it as shown below:

```
int factorial ( int ) ;
int prime ( int ) ;
void fibonacci ( int ) ;
```

- (c) From the Options menu select the menu-item 'Application'. From the dialog that pops up select the option 'Library'. Select OK.

- (d) Compile the program using Alt F9. This would create the library file called 'myfuncs.lib'.

That's it. The library now stands created. Now we have to use the functions defined in this library. Here is how it can be done.

- (a) Create a file, say 'sample.c' and type the following code in it.

```
#include "myfuncs.h"
main()
{
    int f, result ;
    f = factorial ( 5 ) ;
    result = prime ( 13 ) ;
    fibonacci ( 6 ) ;
    printf ( "\n%d %d", f, result ) ;
}
```

Note that the file 'myfuncs.h' should be in the same directory as the file 'sample.c'. If not, then while including 'myfuncs.h' mention the appropriate path.

- (b) Go to the 'Project' menu and select 'Open Project...' option. On doing so a dialog would pop up. Give the name of the project, say 'sample.prj' and select OK.
- (c) From the 'Project' menu select 'Add Item'. On doing so a file dialog would appear. Select the file 'sample.c' and then select 'Add'. Also add the file 'myfuncs.lib' in the same manner. Finally select 'Done'.
- (d) Compile and execute the project using Ctrl F9.

Summary

- (a) To avoid repetition of code and bulky programs functionally related statements are isolated into a function.
- (b) Function declaration specifies what is the return type of the function and the types of parameters it accepts.
- (c) Function definition defines the body of the function.
- (d) Variables declared in a function are not available to other functions in a program. So, there won't be any clash even if we give same name to the variables declared in different functions.
- (e) Pointers are variables which hold addresses of other variables.
- (f) A function can be called either by value or by reference.
- (g) Pointers can be used to make a function return more than one value simultaneously.
- (h) Recursion is difficult to understand, but in some cases offer a better solution than loops.
- (i) Adding too many functions and calling them frequently may slow down the program execution.

Exercise

Simple functions, Passing values between functions

[A] What would be the output of the following programs:

```
(a)  main( )
      {
          printf ( "\nOnly stupids use C?" );
          display( ) ;
      }
      display( )
      {
          printf ( "\nFools too use C!" );
          main( ) ;
      }
```



```
(b)  main()  
    {  
        printf ( "\nC to it that C survives" );  
        main();  
    }
```

```
(c)  main()  
    {  
        int i = 45, c ;  
        c = check ( i );  
        printf ( "\n%d", c );  
    }  
check ( int ch )  
{  
    if ( ch >= 45 )  
        return ( 100 );  
    else  
        return ( 10 * 10 );  
}
```

```
(d)  main()  
    {  
        int i = 45, c ;  
        c = multiply ( i * 1000 );  
        printf ( "\n%d", c );  
    }  
check ( int ch )  
{  
    if ( ch >= 40000 )  
        return ( ch / 10 );  
    else  
        return ( 10 );  
}
```

[B] Point out the errors, if any, in the following programs:

```
(a)  main()  
    {
```

```
    int i = 3, j = 4, k, l ;
    k = addmult ( i, j ) ;
    l = addmult ( i, j ) ;
    printf ( "\n%d %d", k, l ) ;
}
addmult ( int ii, int jj )
{
    int kk, ll ;
    kk = ii + jj ;
    ll = ii * jj ;
    return ( kk, ll ) ;
}

(b) main( )
{
    int a ;
    a = message( ) ;
}
message( )
{
    printf ( "\nViruses are written in C" ) ;
    return ;
}

(c) main( )
{
    float a = 15.5 ;
    char ch = 'C' ;
    printit ( a, ch ) ;
}
printit ( a, ch )
{
    printf ( "\n%f %c", a, ch ) ;
}

(d) main( )
{
    message( ) ;
```

```

    message();
}
message();
{
    printf ( "\nPraise worthy and C worthy are synonyms" );
}

```

(e) `main()`

```

{
    let_us_c()
    {
        printf ( "\nC is a Cimple minded language !" );
        printf ( "\nOthers are of course no match !" );
    }
}

```

(f) `main()`

```

{
    message( message ( ) );
}
void message()
{
    printf ( "\nPraise worthy and C worthy are synonyms" );
}

```

[C] Answer the following:

(a) Is this a correctly written function:

```

sqr ( a );
int a;
{
    return ( a * a );
}

```

(b) State whether the following statements are True or False:

1. The variables commonly used in C functions are available to all the functions in a program.
2. To return the control back to the calling function we must use the keyword **return**.
3. The same variable names can be used in different functions without any conflict.
4. Every called function must contain a **return** statement.
5. A function may contain more than one **return** statements.
6. Each **return** statement in a function may return a different value.
7. A function can still be useful even if you don't pass any arguments to it and the function doesn't return any value back.
8. Same names can be used for different functions without any conflict.
9. A function may be called more than once from any other function.
10. It is necessary for a function to return some value.

[D] Answer the following:

- (a) Write a function to calculate the factorial value of any integer entered through the keyboard.
- (b) Write a function **power (a, b)**, to calculate the value of **a** raised to **b**.

- (c) Write a general-purpose function to convert any given year into its roman equivalent. The following table shows the roman equivalent of decimal numbers:

Decimal	Roman	Decimal	Roman
1	i	100	c
5	v	500	d
10	x	1000	m
50	l		

Example:

Roman equivalent of 1988 is mdcccclxxxviii

Roman equivalent of 1525 is mdxxv

- (d) Any year is entered through the keyboard. Write a function to determine whether the year is a leap year or not.
- (e) A positive integer is entered through the keyboard. Write a function to obtain the prime factors of this number.

For example, prime factors of 24 are 2, 2, 2 and 3, whereas prime factors of 35 are 5 and 7.

Function Prototypes, Call by Value/Reference, Pointers

[E] What would be the output of the following programs:

- (a)

```
main()
{
    float area ;
    int radius = 1 ;
    area = circle ( radius ) ;
    printf ( "\n%f", area ) ;
}
circle ( int r )
```

```
{
    float a ;
    a = 3.14 * r * r ;
    return ( a ) ;
}

(b) main()
{
    void slogan() ;
    int c = 5 ;
    c = slogan() ;
    printf ( "\n%d", c ) ;
}

void slogan()
{
    printf ( "\nOnly He men use C!" ) ;
}
```

[F] Answer the following:

- (a) Write a function which receives a **float** and an **int** from **main()**, finds the product of these two and returns the product which is printed through **main()**.
- (b) Write a function that receives 5 integers and returns the sum, average and standard deviation of these numbers. Call this function from **main()** and print the results in **main()**.
- (c) Write a function that receives marks received by a student in 3 subjects and returns the average and percentage of these marks. Call this function from **main()** and print the results in **main()**.

[G] What would be the output of the following programs:

```
(a) main()
{
    int i = 5, j = 2 ;
```

```
junk ( i, j ) ;  
printf ( "\n%d %d", i, j ) ;  
}  
junk ( int i, int j )  
{  
    i = i * i ;  
    j = j * j ;  
}
```

```
(b) main()  
{  
    int i = 5, j = 2 ;  
    junk ( &i, &j ) ;  
    printf ( "\n%d %d", i, j ) ;  
}  
junk ( int *i, int *j )  
{  
    *i = *i * *i ;  
    *j = *j * *j ;  
}
```

```
(c) main()  
{  
    int i = 4, j = 2 ;  
    junk ( &i, j ) ;  
    printf ( "\n%d %d", i, j ) ;  
}  
junk ( int *i, int j )  
{  
    *i = *i * *i ;  
    j = j * j ;  
}
```

```
(d) main()  
{  
    float a = 13.5 ;  
    float *b, *c ;  
    b = &a ; /* suppose address of a is 1006 */
```

```
c = b ;
printf ( "\n%u %u %u", &a, b, c ) ;
printf ( "\n%f %f %f %f %f", a, *(&a), *&a, *b, *c ) ;
}
```

[H] Point out the errors, if any, in the following programs:

- (a)

```
main( )
{
    int i = 135, a = 135, k ;
    k = pass ( i, a ) ;
    printf ( "\n%d", k ) ;
}
pass ( int j, int b )
int c ;
{
    c = j + b ;
    return ( c ) ;
}
```
- (b)

```
main( )
{
    int p = 23, f = 24 ;
    jiaayjo ( &p, &f ) ;
    printf ( "\n%d %d", p, f ) ;
}
jiaayjo ( int q, int g )
{
    q = q + q ;
    g = g + g ;
}
```
- (c)

```
main( )
{
    int k = 35, z ;
    z = check ( k ) ;
    printf ( "\n%d", z ) ;
}
```



```
check ( m )
{
    int m ;
    if ( m > 40 )
        return ( 1 ) ;
    else
        return ( 0 ) ;
}
```

```
(d) main()
{
    int i = 35, *z ;
    z = function ( &i ) ;
    printf ( "\n%d", z ) ;
}
function ( int *m )
{
    return ( m + 2 ) ;
}
```

[I] What would be the output of the following programs:

```
(a) main()
{
    int i = 0 ;
    i++ ;
    if ( i <= 5 )
    {
        printf ( "\nC adds wings to your thoughts" ) ;
        exit() ;
        main() ;
    }
}
```

```
(b) main()
{
    static int i = 0 ;
    i++ ;
```

```
    if ( i <= 5 )
    {
        printf ( "\n%d", i );
        main();
    }
    else
        exit();
}
```

[J] Attempt the following:

- (a) A 5-digit positive integer is entered through the keyboard, write a function to calculate sum of digits of the 5-digit number:
 - (1) Without using recursion
 - (2) Using recursion
- (b) A positive integer is entered through the keyboard, write a program to obtain the prime factors of the number. Modify the function suitably to obtain the prime factors recursively.
- (c) Write a recursive function to obtain the first 25 numbers of a Fibonacci sequence. In a Fibonacci sequence the sum of two successive terms gives the third term. Following are the first few terms of the Fibonacci sequence:
1 1 2 3 5 8 13 21 34 55 89...
- (d) A positive integer is entered through the keyboard, write a function to find the binary equivalent of this number using recursion.
- (e) Write a recursive function to obtain the running sum of first 25 natural numbers.
- (f) Write a C function to evaluate the series

$$\sin(x) = x - (x^3 / 3!) + (x^5 / 5!) - (x^7 / 7!) + \dots$$

to five significant digits.

- (g) Given three variables **x**, **y**, **z** write a function to circularly shift their values to right. In other words if $x = 5$, $y = 8$, $z = 10$ after circular shift $y = 5$, $z = 8$, $x = 10$ after circular shift $y = 5$, $z = 8$ and $x = 10$. Call the function with variables **a**, **b**, **c** to circularly shift values.
- (h) Write a function to find the binary equivalent of a given decimal integer and display it.
- (i) If the lengths of the sides of a triangle are denoted by **a**, **b**, and **c**, then area of triangle is given by

$$area = \sqrt{S(S-a)(S-b)(S-c)}$$

where, $S = (a + b + c) / 2$

- (j) Write a function to compute the distance between two points and use it to develop another function that will compute the area of the triangle whose vertices are **A(x1, y1)**, **B(x2, y2)**, and **C(x3, y3)**. Use these functions to develop a function which returns a value 1 if the point **(x, y)** lies inside the triangle ABC, otherwise a value 0.
- (k) Write a function to compute the greatest common divisor given by Euclid's algorithm, exemplified for $J = 1980$, $K = 1617$ as follows:

$1980 / 1617 = 1$	$1980 - 1 * 1617 = 363$
$1617 / 363 = 4$	$1617 - 4 * 363 = 165$
$363 / 165 = 2$	$363 - 2 * 165 = 33$
$5 / 33 = 5$	$165 - 5 * 33 = 0$

Thus, the greatest common divisor is 33.

6 *Data Types Revisited*

- Integers, *long* and *short*
- Integers, *signed* and *unsigned*
- Chars, *signed* and *unsigned*
- Floats and Doubles
- A Few More Issues...
- Storage Classes in C
 - Automatic Storage Class
 - Register Storage Class
 - Static Storage Class
 - External Storage Class
 - Which to Use When
- Summary
- Exercise

As seen in the first chapter the primary data types could be of three varieties—**char**, **int**, and **float**. It may seem odd to many, how C programmers manage with such a tiny set of data types. Fact is, the C programmers aren't really deprived. They can derive many data types from these three types. In fact, the number of data types that can be derived in C, is in principle, unlimited. A C programmer can always invent whatever data type he needs.

Not only this, the primary data types themselves could be of several types. For example, a **char** could be an **unsigned char** or a **signed char**. Or an **int** could be a **short int** or a **long int**. Sufficiently confusing? Well, let us take a closer look at these variations of primary data types in this chapter.

To fully define a variable one needs to mention not only its type but also its storage class. In this chapter we would be exploring the different storage classes and their relevance in C programming.

Integers, *long* and *short*

We had seen earlier that the range of an Integer constant depends upon the compiler. For a 16-bit compiler like Turbo C or Turbo C++ the range is -32768 to 32767. For a 32-bit compiler the range would be -2147483648 to +2147483647. Here a 16-bit compiler means that when it compiles a C program it generates machine language code that is targeted towards working on a 16-bit microprocessor like Intel 8086/8088. As against this, a 32-bit compiler like VC++ generates machine language code that is targeted towards a 32-bit microprocessor like Intel Pentium. Note that this does not mean that a program compiled using Turbo C would not work on 32-bit processor. It would run successfully but at that time the 32-bit processor would work as if it were a 16-bit processor. This happens because a 32-bit processor provides support for programs compiled using 16-bit compilers. If this backward compatibility support is not provided the 16-bit program

would not run on it. This is precisely what happens on the new Intel Itanium processors, which have withdrawn support for 16-bit code.

Remember that out of the two/four bytes used to store an integer, the highest bit ($16^{\text{th}}/32^{\text{nd}}$ bit) is used to store the sign of the integer. This bit is 1 if the number is negative, and 0 if the number is positive.

C offers a variation of the integer data type that provides what are called **short** and **long** integer values. The intention of providing these variations is to provide integers with different ranges wherever possible. Though not a rule, **short** and **long** integers would usually occupy two and four bytes respectively. Each compiler can decide appropriate sizes depending on the operating system and hardware for which it is being written, subject to the following rules:

- (a) **shorts** are at least 2 bytes big
- (b) **longs** are at least 4 bytes big
- (c) **shorts** are never bigger than **ints**
- (d) **ints** are never bigger than **longs**

Figure 6.1 shows the sizes of different integers based upon the OS used.

Compiler	short	int	long
16-bit (Turbo C/C++)	2	2	4
32-bit (Visual C++)	2	4	4

Figure 6.1

long variables which hold **long** integers are declared using the keyword **long**, as in,

```
long int i ;  
long int abc ;
```

long integers cause the program to run a bit slower, but the range of values that we can use is expanded tremendously. The value of a **long** integer typically can vary from -2147483648 to +2147483647. More than this you should not need unless you are taking a world census.

If there are such things as **longs**, symmetry requires **shorts** as well—integers that need less space in memory and thus help speed up program execution. **short** integer variables are declared as,

```
short int j ;  
short int height ;
```

C allows the abbreviation of **short int** to **short** and of **long int** to **long**. So the declarations made above can be written as,

```
long i ;  
long abc ;  
short j ;  
short height ;
```

Naturally, most C programmers prefer this short-cut.

Sometimes we come across situations where the constant is small enough to be an **int**, but still we want to give it as much storage as a **long**. In such cases we add the suffix 'L' or 'l' at the end of the number, as in 23L.

Integers, signed and unsigned

Sometimes, we know in advance that the value stored in a given integer variable will always be positive—when it is being used to

only count things, for example. In such a case we can declare the variable to be **unsigned**, as in,

```
unsigned int num_students ;
```

With such a declaration, the range of permissible integer values (for a 16-bit OS) will shift from the range -32768 to +32767 to the range 0 to 65535. Thus, declaring an integer as **unsigned** almost doubles the size of the largest possible value that it can otherwise take. This so happens because on declaring the integer as **unsigned**, the left-most bit is now free and is not used to store the sign of the number. Note that an **unsigned** integer still occupies two bytes. This is how an **unsigned** integer can be declared:

```
unsigned int i ;  
unsigned i ;
```

Like an **unsigned int**, there also exists a **short unsigned int** and a **long unsigned int**. By default a **short int** is a **signed short int** and a **long int** is a **signed long int**.

Chars, *signed* and *unsigned*

Parallel to **signed** and **unsigned ints** (either **short** or **long**), similarly there also exist **signed** and **unsigned chars**, both occupying one byte each, but having different ranges. To begin with it might appear strange as to how a **char** can have a sign. Consider the statement

```
char ch = 'A' ;
```

Here what gets stored in **ch** is the binary equivalent of the ASCII value of 'A' (i.e. binary of 65). And if 65's binary can be stored, then -54's binary can also be stored (in a **signed char**).

A **signed char** is same as an ordinary **char** and has a range from -128 to +127; whereas, an **unsigned char** has a range from 0 to 255. Let us now see a program that illustrates this range:

```
main()
{
    char ch = 291 ;
    printf ( "\n%d %c", ch, ch ) ;
}
```

What output do you expect from this program? Possibly, 291 and the character corresponding to it. Well, not really. Surprised? The reason is that **ch** has been defined as a **char**, and a **char** cannot take a value bigger than +127. Hence when value of **ch** exceeds +127, an appropriate value from the other side of the range is picked up and stored in **ch**. This value in our case happens to be 35, hence 35 and its corresponding character #, gets printed out.

Here is another program that would make the concept clearer.

```
main()
{
    char ch ;

    for ( ch = 0 ; ch <= 255 ; ch++ )
        printf ( "\n%d %c", ch, ch ) ;
}
```

This program should output ASCII values and their corresponding characters. Well, No! This is an indefinite loop. The reason is that **ch** has been defined as a **char**, and a **char** cannot take values bigger than +127. Hence when value of **ch** is +127 and we perform **ch++** it becomes -128 instead of +128. -128 is less than 255 hence the condition is still satisfied. Here onwards **ch** would take values like -127, -126, -125, -2, -1, 0, +1, +2, ... +127, -128, -127, etc. Thus the value of **ch** would keep oscillating between -128 to +127, thereby ensuring that the loop never gets terminated. How do you

overcome this difficulty? Would declaring **ch** as an **unsigned char** solve the problem? Even this would not serve the purpose since when **ch** reaches a value 255, **ch++** would try to make it 256 which cannot be stored in an **unsigned char**. Thus the only alternative is to declare **ch** as an **int**. However, if we are bent upon writing the program using **unsigned char**, it can be done as shown below. The program is definitely less elegant, but workable all the same.

```
main()  
{  
    unsigned char ch ;  
  
    for ( ch = 0 ; ch <= 254 ; ch++ )  
        printf ( "\n%d %c", ch, ch ) ;  
  
    printf ( "\n%d %c", ch, ch ) ;  
}
```

Floats and Doubles

A **float** occupies four bytes in memory and can range from -3.4e38 to +3.4e38. If this is insufficient then C offers a **double** data type that occupies 8 bytes in memory and has a range from -1.7e308 to +1.7e308. A variable of type **double** can be declared as,

```
double a, population ;
```

If the situation demands usage of real numbers that lie even beyond the range offered by **double** data type, then there exists a **long double** that can range from -1.7e4932 to +1.7e4932. A **long double** occupies 10 bytes in memory.

You would see that most of the times in C programming one is required to use either **chars** or **ints** and cases where **floats**, **doubles** or **long doubles** would be used are indeed rare.

Let us now write a program that puts to use all the data types that we have learnt in this chapter. Go through the following program carefully, which shows how to use these different data types. Note the format specifiers used to input and output these data types.

```
main()
{
    char c ;
    unsigned char d ;
    int i ;
    unsigned int j ;
    short int k ;
    unsigned short int l ;
    long int m ;
    unsigned long int n ;
    float x ;
    double y ;
    long double z ;

    /* char */
    scanf ( "%c %c", &c, &d ) ;
    printf ( "%c %c", c, d ) ;

    /* int */
    scanf ( "%d %u", &i, &j ) ;
    printf ( "%d %u", i, j ) ;

    /* short int */
    scanf ( "%d %u", &k, &l ) ;
    printf ( "%d %u", k, l ) ;

    /* long int */
    scanf ( "%ld %lu", &m, &n ) ;
    printf ( "%ld %lu", m, n ) ;

    /* float, double, long double */
    scanf ( "%f %lf %Lf", &x, &y, &z ) ;
    printf ( "%f %lf %Lf", x, y, z ) ;
```

}

The essence of all the data types that we have learnt so far has been captured in Figure 6.2.

Data Type	Range	Bytes	Format
signed char	-128 to + 127	1	%c
unsigned char	0 to 255	1	%c
short signed int	-32768 to +32767	2	%d
short unsigned int	0 to 65535	2	%u
signed int	-32768 to +32767	2	%d
unsigned int	0 to 65535	2	%u
long signed int	-2147483648 to +2147483647	4	%ld
long unsigned int	0 to 4294967295	4	%lu
float	-3.4e38 to +3.4e38	4	%f
double	-1.7e308 to +1.7e308	8	%lf
long double	-1.7e4932 to +1.7e4932	10	%Lf
Note: The sizes and ranges of int, short and long are compiler dependent. Sizes in this figure are for 16-bit compiler.			

Figure 6.2

A Few More Issues...

Having seen all the variations of the primary types let us take a look at some more related issues.

- (a) We saw earlier that size of an integer is compiler dependent. This is even true in case of **chars** and **floats**. Also, depending upon the microprocessor for which the compiler targets its code the accuracy of floating point calculations may change. For example, the result of 22.0/7.0 would be reported more

accurately by VC++ compiler as compared to TC/TC++ compilers. This is because TC/TC++ targets its compiled code to 8088/8086 (16-bit) microprocessors. Since these microprocessors do not offer floating point support, TC/TC++ performs all float operations using a software piece called Floating Point Emulator. This emulator has limitations and hence produces less accurate results. Also, this emulator becomes part of the EXE file, thereby increasing its size. In addition to this increased size there is a performance penalty since this bigger code would take more time to execute.

- (b) If you look at ranges of **chars** and **ints** there seems to be one extra number on the negative side. This is because a negative number is always stored as 2's compliment of its binary. For example, let us see how -128 is stored. Firstly, binary of 128 is calculated (10000000), then its 1's compliment is obtained (01111111). A 1's compliment is obtained by changing all 0s to 1s and 1s to 0s. Finally, 2's compliment of this number, i.e. 10000000, gets stored. A 2's compliment is obtained by adding 1 to the 1's compliment. Thus, for -128, 10000000 gets stored. This is an 8-bit number and it can be easily accommodated in a **char**. As against this, +128 cannot be stored in a **char** because its binary 010000000 (left-most 0 is for positive sign) is a 9-bit number. However +127 can be stored as its binary 01111111 turns out to be a 8-bit number.
- (c) What happens when we attempt to store +128 in a **char**? The first number on the negative side, i.e. -128 gets stored. This is because from the 9-bit binary of +128, 010000000, only the right-most 8 bits get stored. But when 10000000 is stored the left-most bit is 1 and it is treated as a sign bit. Thus the value of the number becomes -128 since it is indeed the binary of -128, as can be understood from (b) above. Similarly, you can verify that an attempt to store +129 in a **char** results in storing -127 in it. In general, if we exceed the range from positive side we end up on the negative side. Vice versa is

also true. If we exceed the range from negative side we end up on positive side.

Storage Classes in C

We have already said all that needs to be said about constants, but we are not finished with variables. To fully define a variable one needs to mention not only its ‘type’ but also its ‘storage class’. In other words, not only do all variables have a data type, they also have a ‘storage class’.

We have not mentioned storage classes yet, though we have written several programs in C. We were able to get away with this because storage classes have defaults. If we don’t specify the storage class of a variable in its declaration, the compiler will assume a storage class depending on the context in which the variable is used. Thus, variables have certain default storage classes.

From C compiler’s point of view, a variable name identifies some physical location within the computer where the string of bits representing the variable’s value is stored. There are basically two kinds of locations in a computer where such a value may be kept—Memory and CPU registers. It is the variable’s storage class that determines in which of these two locations the value is stored.

Moreover, a variable’s storage class tells us:

- (a) Where the variable would be stored.
- (b) What will be the initial value of the variable, if initial value is not specifically assigned.(i.e. the default initial value).
- (c) What is the scope of the variable; i.e. in which functions the value of the variable would be available.
- (d) What is the life of the variable; i.e. how long would the variable exist.

There are four storage classes in C:

- (a) Automatic storage class
- (b) Register storage class
- (c) Static storage class
- (d) External storage class

Let us examine these storage classes one by one.

Automatic Storage Class

The features of a variable defined to have an automatic storage class are as under:

Storage	– Memory.
Default initial value	– An unpredictable value, which is often called a garbage value.
Scope	– Local to the block in which the variable is defined.
Life	– Till the control remains within the block in which the variable is defined.

Following program shows how an automatic storage class variable is declared, and the fact that if the variable is not initialized it contains a garbage value.

```
main()
{
    auto int i, j;
    printf ( "\n%d %d", i, j );
}
```

The output of the above program could be...

1211 221

where, 1211 and 221 are garbage values of **i** and **j**. When you run this program you may get different values, since garbage values

are unpredictable. So always make it a point that you initialize the automatic variables properly, otherwise you are likely to get unexpected results. Note that the keyword for this storage class is **auto**, and not **automatic**.

Scope and life of an automatic variable is illustrated in the following program.

```
main()
{
    auto int i = 1 ;
    {
        {
            {
                printf ( "\n%d ", i );
            }
            printf ( "%d ", i );
        }
        printf ( "%d", i );
    }
}
```

The output of the above program is:

1 1 1

This is because, all **printf()** statements occur within the outermost block (a block is all statements enclosed within a pair of braces) in which **i** has been defined. It means the scope of **i** is local to the block in which it is defined. The moment the control comes out of the block in which the variable is defined, the variable and its value is irretrievably lost. To catch my point, go through the following program.

```
main()
{
    auto int i = 1 ;
    {
```



```
    auto int i = 2 ;
    {
        auto int i = 3 ;
        printf ( "\n%d ", i ) ;
    }
    printf ( "%d ", i ) ;
}
printf ( "%d", i ) ;
}
```

The output of the above program would be:

3 2 1

Note that the Compiler treats the three **i**'s as totally different variables, since they are defined in different blocks. Once the control comes out of the innermost block the variable **i** with value 3 is lost, and hence the **i** in the second **printf()** refers to **i** with value 2. Similarly, when the control comes out of the next innermost block, the third **printf()** refers to the **i** with value 1.

Understand the concept of life and scope of an automatic storage class variable thoroughly before proceeding with the next storage class.

Register Storage Class

The features of a variable defined to be of **register** storage class are as under:

Storage	- CPU registers.
Default initial value	- Garbage value.
Scope	- Local to the block in which the variable is defined.
Life	- Till the control remains within the block in which the variable is defined.

A value stored in a CPU register can always be accessed faster than the one that is stored in memory. Therefore, if a variable is used at many places in a program it is better to declare its storage class as **register**. A good example of frequently used variables is loop counters. We can name their storage class as **register**.

```
main()  
{  
    register int i ;  
  
    for ( i = 1 ; i <= 10 ; i++ )  
        printf ( "\n%d", i ) ;  
}
```

Here, even though we have declared the storage class of **i** as **register**, we cannot say for sure that the value of **i** would be stored in a CPU register. Why? Because the number of CPU registers are limited, and they may be busy doing some other task. What happens in such an event... the variable works as if its storage class is **auto**.

Not every type of variable can be stored in a CPU register.

For example, if the microprocessor has 16-bit registers then they cannot hold a **float** value or a **double** value, which require 4 and 8 bytes respectively. However, if you use the **register** storage class for a **float** or a **double** variable you won't get any error messages. All that would happen is the compiler would treat the variables to be of **auto** storage class.

Static Storage Class

The features of a variable defined to have a **static** storage class are as under:

- | | |
|-----------------------|-----------|
| Storage | – Memory. |
| Default initial value | – Zero. |

Scope	– Local to the block in which the variable is defined.
Life	– Value of the variable persists between different function calls.

Compare the two programs and their output given in Figure 6.3 to understand the difference between the **automatic** and **static** storage classes.

<pre>main() { increment(); increment(); increment(); } increment() { auto int i = 1; printf ("%d\n", i); i = i + 1; }</pre>	<pre>main() { increment(); increment(); increment(); } increment() { static int i = 1; printf ("%d\n", i); i = i + 1; }</pre>
The output of the above programs would be:	
<pre>1 1 1</pre>	<pre>1 2 3</pre>

Figure 6.3

The programs above consist of two functions **main()** and **increment()**. The function **increment()** gets called from **main()** thrice. Each time it increments the value of **i** and prints it. The only difference in the two programs is that one uses an **auto** storage class for variable **i**, whereas the other uses **static** storage class.

Like **auto** variables, **static** variables are also local to the block in which they are declared. The difference between them is that **static** variables don't disappear when the function is no longer active. Their values persist. If the control comes back to the same function again the **static** variables have the same values they had last time around.

In the above example, when variable **i** is **auto**, each time **increment()** is called it is re-initialized to one. When the function terminates, **i** vanishes and its new value of 2 is lost. The result: no matter how many times we call **increment()**, **i** is initialized to 1 every time.

On the other hand, if **i** is **static**, it is initialized to 1 only once. It is never initialized again. During the first call to **increment()**, **i** is incremented to 2. Because **i** is static, this value persists. The next time **increment()** is called, **i** is not re-initialized to 1; on the contrary its old value 2 is still available. This current value of **i** (i.e. 2) gets printed and then **i = i + 1** adds 1 to **i** to get a value of 3. When **increment()** is called the third time, the current value of **i** (i.e. 3) gets printed and once again **i** is incremented. In short, if the storage class is **static** then the statement **static int i = 1** is executed only once, irrespective of how many times the same function is called.

Consider one more program.

```
main()
{
    int *j;
    int *fun( );
    j = fun( );
    printf ( "\n%d", *j );
}

int *fun( )
{
```

```
int k = 35 ;  
return ( &k );  
}
```

Here we are returning an address of **k** from **fun()** and collecting it in **j**. Thus **j** becomes pointer to **k**. Then using this pointer we are printing the value of **k**. This correctly prints out 35. Now try calling any function (even **printf()**) immediately after the call to **fun()**. This time **printf()** prints a garbage value. Why does this happen? In the first case, when the control returned from **fun()** though **k** went dead it was still left on the stack. We then accessed this value using its address that was collected in **j**. But when we precede the call to **printf()** by a call to any other function, the stack is now changed, hence we get the garbage value. If we want to get the correct value each time then we must declare **k** as **static**. By doing this when the control returns from **fun()**, **k** would not die.

All this having been said, a word of advice—avoid using **static** variables unless you really need them. Because their values are kept in memory when the variables are not active, which means they take up space in memory that could otherwise be used by other variables.

External Storage Class

The features of a variable whose storage class has been defined as external are as follows:

Storage	– Memory.
Default initial value	– Zero.
Scope	– Global.
Life	– As long as the program's execution doesn't come to an end.

External variables differ from those we have already discussed in that their scope is global, not local. External variables are declared outside all functions, yet are available to all functions that care to use them. Here is an example to illustrate this fact.

```
int i ;
main( )
{
    printf ( "\ni = %d", i ) ;

    increment( ) ;
    increment( ) ;
    decrement( ) ;
    decrement( ) ;
}

increment( )
{
    i = i + 1 ;
    printf ( "\non incrementing i = %d", i ) ;
}

decrement( )
{
    i = i - 1 ;
    printf ( "\non decrementing i = %d", i ) ;
}
```

The output would be:

```
i = 0
on incrementing i = 1
on incrementing i = 2
on decrementing i = 1
on decrementing i = 0
```

As is obvious from the above output, the value of **i** is available to the functions **increment()** and **decrement()** since **i** has been declared outside all functions.

Look at the following program.

```
int x = 21 ;
main()
{
    extern int y ;
    printf ( "\n%d %d", x, y ) ;
}
int y = 31 ;
```

Here, **x** and **y** both are global variables. Since both of them have been defined outside all the functions both enjoy external storage class. Note the difference between the following:

```
extern int y ;
int y = 31 ;
```

Here the first statement is a declaration, whereas the second is the definition. When we declare a variable no space is reserved for it, whereas, when we define it space gets reserved for it in memory. We had to declare **y** since it is being used in **printf()** before it's definition is encountered. There was no need to declare **x** since its definition is done before its usage. Also remember that a variable can be declared several times but can be defined only once.

Another small issue—what will be the output of the following program?

```
int x = 10 ;
main()
{
    int x = 20 ;

    printf ( "\n%d", x ) ;
```

```
        display( ) ;  
    }  
    display( )  
    {  
        printf ( "\n%d", x ) ;  
    }
```

Here **x** is defined at two places, once outside **main()** and once inside it. When the control reaches the **printf()** in **main()** which **x** gets printed? Whenever such a conflict arises, it's the local variable that gets preference over the global variable. Hence the **printf()** outputs 20. When **display()** is called and control reaches the **printf()** there is no such conflict. Hence this time the value of the global **x**, i.e. 10 gets printed.

One last thing—a **static** variable can also be declared outside all the functions. For all practical purposes it will be treated as an **extern** variable. However, the scope of this variable is limited to the same file in which it is declared. This means that the variable would not be available to any function that is defined in a file other than the file in which the variable is defined.

Which to Use When

Dennis Ritchie has made available to the C programmer a number of storage classes with varying features, believing that the programmer is in a best position to decide which one of these storage classes is to be used when. We can make a few ground rules for usage of different storage classes in different programming situations with a view to:

- (a) economise the memory space consumed by the variables
- (b) improve the speed of execution of the program

The rules are as under:

- Use **static** storage class only if you want the value of a variable to persist between different function calls.
- Use **register** storage class for only those variables that are being used very often in a program. Reason is, there are very few CPU registers at our disposal and many of them might be busy doing something else. Make careful utilization of the scarce resources. A typical application of **register** storage class is loop counters, which get used a number of times in a program.
- Use **extern** storage class for only those variables that are being used by almost all the functions in the program. This would avoid unnecessary passing of these variables as arguments when making a function call. Declaring all the variables as **extern** would amount to a lot of wastage of memory space because these variables would remain active throughout the life of the program.
- If you don't have any of the express needs mentioned above, then use the **auto** storage class. In fact most of the times we end up using the **auto** variables, because often it so happens that once we have used the variables in a function we don't mind losing them.

Summary

- (a) We can use different variations of the primary data types, namely **signed** and **unsigned char**, **long** and **short int**, **float**, **double** and **long double**. There are different format specifications for all these data types when they are used in **scanf()** and **printf()** functions.
- (b) The maximum value a variable can hold depends upon the number of bytes it occupies in memory.
- (c) By default all the variables are **signed**. We can declare a variable as **unsigned** to accommodate greater value without increasing the bytes occupied.

- (d) We can make use of proper storage classes like **auto**, **register**, **static** and **extern** to control four properties of the variable—storage, default initial value, scope and life.

Exercise

[A] What would be the output of the following programs:

- (a)

```
main()
{
    int i ;
    for ( i = 0 ; i <= 50000 ; i++ )
        printf ( "\n%d", i ) ;
}
```
- (b)

```
main()
{
    float a = 13.5 ;
    double b = 13.5 ;
    printf ( "\n%f %lf", a, b ) ;
}
```
- (c)

```
int i = 0 ;
main()
{
    printf ( "\nmain's i = %d", i ) ;
    i++ ;
    val() ;
    printf ( "\nmain's i = %d", i ) ;
    val() ;
}
val()
{
    i = 100 ;
    printf ( "\nval's i = %d", i ) ;
    i++ ;
}
```

```
(d)  main()  
    {  
        int x, y, s = 2 ;  
        s *= 3 ;  
        y = f ( s ) ;  
        x = g ( s ) ;  
        printf ( "\n%d %d %d", s, y, x ) ;  
    }  
    int t = 8 ;  
    f ( int a )  
    {  
        a += -5 ;  
        t -= 4 ;  
        return ( a + t ) ;  
    }  
    g ( int a )  
    {  
        a = 1 ;  
        t += a ;  
        return ( a + t ) ;  
    }  
}
```

```
(e)  main()  
    {  
        static int count = 5 ;  
        printf ( "\ncount = %d", count-- ) ;  
        if ( count != 0 )  
            main() ;  
    }
```

```
(f)  main()  
    {  
        int i, j ;  
        for ( i = 1 ; i < 5 ; i++ )  
        {  
            j = g ( i ) ;  
            printf ( "\n%d", j ) ;  
        }  
    }
```

```
    }  
    g ( int x )  
    {  
        static int v = 1 ;  
        int b = 3 ;  
        v += x ;  
        return ( v + x + b ) ;  
    }  
  
(g) float x = 4.5 ;  
    main()  
    {  
        float y, float f ( float ) ;  
        x *= 2.0 ;  
        y = f ( x ) ;  
        printf ( "\n%f %f", x, y ) ;  
    }  
    float f ( float a )  
    {  
        a += 1.3 ;  
        x -= 4.5 ;  
        return ( a + x ) ;  
    }  
  
(h) main()  
    {  
        func() ;  
        func() ;  
    }  
    func()  
    {  
        auto int i = 0 ;  
        register int j = 0 ;  
        static int k = 0 ;  
        i++ ; j++ ; k++ ;  
        printf ( "\n %d %d %d", i, j, k ) ;  
    }
```

```
(i)  int x = 10 ;
      main( )
      {
          int x = 20 ;
          {
              int x = 30 ;
              printf ( "\n%d", x ) ;
          }
          printf ("\n%d", x ) ;
      }
```

[B] Point out the errors, if any, in the following programs:

- ```
(a) main()
 {
 long num ;
 num = 2 ;
 printf ("\n%d", num) ;
 }
```
- ```
(b)  main( )
      {
          char ch = 200 ;
          printf ( "\n%d", ch ) ;
      }
```
- ```
(c) main()
 {
 unsigned a = 25 ;
 long unsigned b = 25l ;
 printf ("\n%lu %u", a, b) ;
 }
```
- ```
(d)  main( )
      {
          long float a = 25.345e454 ;
          unsigned double b = 25 ;
          printf ( "\n%lf %d", a, b ) ;
      }
```

```
    }  
(e)  main( )  
    {  
        float a = 25.345 ;  
        float *b ;  
        b = &a ;  
        printf ( "\n%f %u", a, b ) ;  
    }  
(f)  static int y ;  
    main( )  
    {  
        static int z ;  
        printf ("%d %d", y, z ) ;  
    }
```

[C] State whether the following statements are True or False:

- (a) Storage for a register storage class variable is allocated each time the control reaches the block in which it is present.
- (b) An extern storage class variable is not available to the functions that precede its definition, unless the variable is explicitly declared in these functions.
- (c) The value of an automatic storage class variable persists between various function invocations.
- (d) If the CPU registers are not available, the register storage class variables are treated as static storage class variables.
- (e) The register storage class variables cannot hold float values.
- (f) If we try to use register storage class for a **float** variable the compiler will flash an error message.

- (g) If the variable **x** is defined as **extern** and a variable **x** is also defined as a local variable of some function, then the global variable gets preference over the local variable.
- (h) The default value for automatic variable is zero.
- (i) The life of static variable is till the control remains within the block in which it is defined.
- (j) If a global variable is to be defined, then the **extern** keyword is necessary in its declaration.
- (k) The address of register variable is not accessible.

[D] Following program calculates the sum of digits of the number 12345. Go through it and find out why is it necessary to declare the storage class of the variable **sum** as **static**.

```
main()  
{  
    int a ;  
    a = sumdig ( 12345 ) ;  
    printf ( "\n%d", a ) ;  
}  
sumdig ( int num )  
{  
    static int sum ;  
    int a, b ;  
    a = num % 10 ;  
    b = ( num - a ) / 10 ;  
    sum = sum + a ;  
    if ( b != 0 )  
        sumdig ( b ) ;  
    else  
        return ( sum ) ;  
}
```

7 *The C Preprocessor*

- Features of C Preprocessor
- Macro Expansion
 - Macros with Arguments
 - Macros versus Functions
- File Inclusion
- Conditional Compilation
- *#if* and *#elif* Directives
- Miscellaneous Directives
 - *#undef* Directive
 - *#pragma* Directive
- Summary
- Exercise

The C preprocessor is exactly what its name implies. It is a program that processes our source program before it is passed to the compiler. Preprocessor commands (often known as directives) form what can almost be considered a language within C language. We can certainly write C programs without knowing anything about the preprocessor or its facilities. But preprocessor is such a great convenience that virtually all C programmers rely on it. This chapter explores the preprocessor directives and discusses the pros and cons of using them in programs.

Features of C Preprocessor

There are several steps involved from the stage of writing a C program to the stage of getting it executed. Figure 7.1 shows these different steps along with the files created during each stage. You can observe from the figure that our program passes through several processors before it is ready to be executed. The input and output to each of these processors is shown in Figure 7.2.

Note that if the source code is stored in a file PR1.C then the expanded source code gets stored in a file PR1.I. When this expanded source code is compiled the object code gets stored in PR1.OBJ. When this object code is linked with the object code of library functions the resultant executable code gets stored in PR1.EXE.

The preprocessor offers several features called preprocessor directives. Each of these preprocessor directives begin with a # symbol. The directives can be placed anywhere in a program but are most often placed at the beginning of a program, before the first function definition. We would learn the following preprocessor directives here:

- (a) Macro expansion
- (b) File inclusion

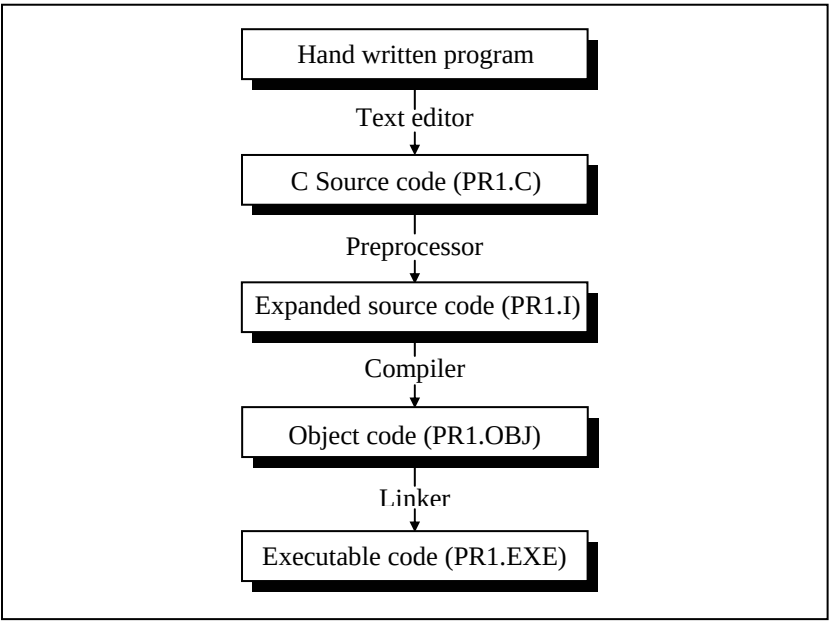


Figure 7.1

Processor	Input	Output
Editor	Program typed from keyboard	C source code containing program and preprocessor commands
Preprocessor	C source code file	Source code file with the preprocessing commands properly sorted out
Compiler	Source code file with preprocessing commands sorted out	Relocatable object code
Linker	Relocatable object code and the standard C library functions	Executable code in machine language

Figure 7.2

- (c) Conditional Compilation
- (d) Miscellaneous directives

Let us understand these features of preprocessor one by one.

Macro Expansion

Have a look at the following program.

```
#define UPPER 25
main()
{
    int i ;
    for ( i = 1 ; i <= UPPER ; i++ )
        printf ( "\n%d", i ) ;
}
```

In this program instead of writing 25 in the **for** loop we are writing it in the form of UPPER, which has already been defined before **main()** through the statement,

```
#define UPPER 25
```

This statement is called ‘macro definition’ or more commonly, just a ‘macro’. What purpose does it serve? During preprocessing, the preprocessor replaces every occurrence of UPPER in the program with 25. Here is another example of macro definition.

```
#define PI 3.1415
main()
{
    float r = 6.25 ;
    float area ;

    area = PI * r * r ;
    printf ( "\nArea of circle = %f", area ) ;
}
```

UPPER and PI in the above programs are often called ‘macro templates’, whereas, 25 and 3.1415 are called their corresponding ‘macro expansions’.

When we compile the program, before the source code passes to the compiler it is examined by the C preprocessor for any macro definitions. When it sees the **#define** directive, it goes through the entire program in search of the macro templates; wherever it finds one, it replaces the macro template with the appropriate macro expansion. Only after this procedure has been completed is the program handed over to the compiler.

In C programming it is customary to use capital letters for macro template. This makes it easy for programmers to pick out all the macro templates when reading through the program.

Note that a macro template and its macro expansion are separated by blanks or tabs. A space between **#** and **define** is optional. Remember that a macro definition is never to be terminated by a semicolon.

And now a million dollar question... why use **#define** in the above programs? What have we gained by substituting PI for 3.1415 in our program? Probably, we have made the program easier to read. Even though 3.1415 is such a common constant that it is easily recognizable, there are many instances where a constant doesn’t reveal its purpose so readily. For example, if the phrase “\x1B[2J” causes the screen to clear. But which would you find easier to understand in the middle of your program “\x1B[2J” or “CLEARSCREEN”? Thus, we would use the macro definition

```
#define CLEARSCREEN "\x1B[2J"
```

Then wherever CLEARSCREEN appears in the program it would automatically be replaced by “\x1B[2J” before compilation begins.

There is perhaps a more important reason for using macro definition than mere readability. Suppose a constant like 3.1415 appears many times in your program. This value may have to be changed some day to 3.141592. Ordinarily, you would need to go through the program and manually change each occurrence of the constant. However, if you have defined PI in a **#define** directive, you only need to make one change, in the **#define** directive itself:

```
#define PI 3.141592
```

Beyond this the change will be made automatically to all occurrences of PI before the beginning of compilation.

In short, it is nice to know that you would be able to change values of a constant at all the places in the program by just making a change in the **#define** directive. This convenience may not matter for small programs shown above, but with large programs macro definitions are almost indispensable.

But the same purpose could have been served had we used a variable **pi** instead of a macro template **PI**. A variable could also have provided a meaningful name for a constant and permitted one change to effect many occurrences of the constant. It's true that a variable can be used in this way. Then, why not use it? For three reasons it's a bad idea.

Firstly, it is inefficient, since the compiler can generate faster and more compact code for constants than it can for variables. Secondly, using a variable for what is really a constant encourages sloppy thinking and makes the program more difficult to understand: if something never changes, it is hard to imagine it as a variable. And thirdly, there is always a danger that the variable may inadvertently get altered somewhere in the program. So it's no longer a constant that you think it is.

Thus, using **#define** can produce more efficient and more easily understandable programs. This directive is used extensively by C programmers, as you will see in many programs in this book.

Following three examples show places where a **#define** directive is popularly used by C programmers.

A **#define** directive is many a times used to define operators as shown below.

```
#define AND &&
#define OR ||
main()
{
    int f = 1, x = 4, y = 90 ;

    if ( ( f < 5 ) AND ( x <= 20 OR y <= 45 ) )
        printf ( "\nYour PC will always work fine..." ) ;
    else
        printf ( "\nIn front of the maintenance man" ) ;
}
```

A **#define** directive could be used even to replace a condition, as shown below.

```
#define AND &&
#define ARANGE ( a > 25 AND a < 50 )
main()
{
    int a = 30 ;

    if ( ARANGE )
        printf ( "within range" ) ;
    else
        printf ( "out of range" ) ;
}
```

A **#define** directive could be used to replace even an entire C statement. This is shown below.

```
#define FOUND printf ( "The Yankee Doodle Virus" );
main()
{
    char signature ;

    if ( signature == 'Y' )
        FOUND
    else
        printf ( "Safe... as yet !" ) ;
}
```

Macros with Arguments

The macros that we have used so far are called simple macros. Macros can have arguments, just as functions can. Here is an example that illustrates this fact.

```
#define AREA(x) ( 3.14 * x * x )
main()
{
    float r1 = 6.25, r2 = 2.5, a ;

    a = AREA ( r1 ) ;
    printf ( "\nArea of circle = %f", a ) ;
    a = AREA ( r2 ) ;
    printf ( "\nArea of circle = %f", a ) ;
}
```

Here's the output of the program...

Area of circle = 122.656250

Area of circle = 19.625000

In this program wherever the preprocessor finds the phrase **AREA(x)** it expands it into the statement (**3.14 * x * x**). However, that's not all that it does. The **x** in the macro template **AREA(x)** is an argument that matches the **x** in the macro expansion (**3.14 * x * x**). The statement **AREA(r1)** in the program causes the variable **r1** to be substituted for **x**. Thus the statement **AREA(r1)** is equivalent to:

```
( 3.14 * r1 * r1 )
```

After the above source code has passed through the preprocessor, what the compiler gets to work on will be this:

```
main()  
{  
    float r1 = 6.25, r2 = 2.5, a ;  
  
    a = 3.14 * r1 * r1 ;  
    printf ( "Area of circle = %f\n", a ) ;  
    a = 3.14 * r2 * r2 ;  
    printf ( "Area of circle = %f", a ) ;  
}
```

Here is another example of macros with arguments:

```
#define ISDIGIT(y) ( y >= 48 && y <= 57 )  
main()  
{  
    char ch ;  
  
    printf ( "Enter any digit " ) ;  
    scanf ( "%c", &ch ) ;  
  
    if ( ISDIGIT ( ch ) )  
        printf ( "\nYou entered a digit" ) ;  
    else  
        printf ( "\nIllegal input" ) ;  
}
```


}

Here are some important points to remember while writing macros with arguments:

- (a) Be careful not to leave a blank between the macro template and its argument while defining the macro. For example, there should be no blank between **AREA** and **(x)** in the definition, `#define AREA(x) ( 3.14 * x * x )`

If we were to write **AREA (x)** instead of **AREA(x)**, the **(x)** would become a part of macro expansion, which we certainly don't want. What would happen is, the template would be expanded to

`( r1 ) ( 3.14 * r1 * r1 )`

which won't run. Not at all what we wanted.

- (b) The entire macro expansion should be enclosed within parentheses. Here is an example of what would happen if we fail to enclose the macro expansion within parentheses.

```
#define SQUARE(n) n * n
main()
{
    int j ;

    j = 64 / SQUARE ( 4 ) ;
    printf ( "j = %d", j ) ;
}
```

The output of the above program would be:

`j = 64`

whereas, what we expected was `j = 4`.

What went wrong? The macro was expanded into

```
j = 64 / 4 * 4 ;
```

which yielded 64.

- (c) Macros can be split into multiple lines, with a ‘\’ (back slash) present at the end of each line. Following program shows how we can define and use multiple line macros.

```
#define HLINE  for ( i = 0 ; i < 79 ; i++ ) \
                printf ( "%c", 196 ) ;

#define VLINE( X, Y ) {\
                    gotoxy ( X, Y ) ; \
                    printf ( "%c", 179 ) ; \
                }

main()
{
    int i, y ;
    clrscr() ;

    gotoxy ( 1, 12 ) ;
    HLINE

    for ( y = 1 ; y < 25 ; y++ )
        VLINE ( 39, y ) ;
}
```

This program draws a vertical and a horizontal line in the center of the screen.

- (d) If for any reason you are unable to debug a macro then you should view the expanded code of the program to see how the macros are getting expanded. If your source code is present in the file PR1.C then the expanded source code would be stored

in PR1.I. You need to generate this file at the command prompt by saying:

```
cpp pr1.c
```

Here CPP stands for C PreProcessor. It generates the expanded source code and stores it in a file called PR1.I. You can now open this file and see the expanded source code. Note that the file PR1.I gets generated in C:\TC\BIN directory. The procedure for generating expanded source code for compilers other than Turbo C/C++ might be a little different.

Macros versus Functions

In the above example a macro was used to calculate the area of the circle. As we know, even a function can be written to calculate the area of the circle. Though macro calls are 'like' function calls, they are not really the same things. Then what is the difference between the two?

In a macro call the preprocessor replaces the macro template with its macro expansion, in a stupid, unthinking, literal way. As against this, in a function call the control is passed to a function along with certain arguments, some calculations are performed in the function and a useful value is returned back from the function.

This brings us to a question: when is it best to use macros with arguments and when is it better to use a function? Usually macros make the program run faster but increase the program size, whereas functions make the program smaller and compact.

If we use a macro hundred times in a program, the macro expansion goes into our source code at hundred different places, thus increasing the program size. On the other hand, if a function is used, then even if it is called from hundred different places in

the program, it would take the same amount of space in the program.

But passing arguments to a function and getting back the returned value does take time and would therefore slow down the program. This gets avoided with macros since they have already been expanded and placed in the source code before compilation.

Moral of the story is—if the macro is simple and sweet like in our examples, it makes nice shorthand and avoids the overheads associated with function calls. On the other hand, if we have a fairly large macro and it is used fairly often, perhaps we ought to replace it with a function.

File Inclusion

The second preprocessor directive we'll explore in this chapter is file inclusion. This directive causes one file to be included in another. The preprocessor command for file inclusion looks like this:

```
#include "filename"
```

and it simply causes the entire contents of **filename** to be inserted into the source code at that point in the program. Of course this presumes that the file being included is existing. When and why this feature is used? It can be used in two cases:

- (a) If we have a very large program, the code is best divided into several different files, each containing a set of related functions. It is a good programming practice to keep different sections of a large program separate. These files are **#included** at the beginning of main program file.
- (b) There are some functions and some macro definitions that we need almost in all programs that we write. These commonly

needed functions and macro definitions can be stored in a file, and that file can be included in every program we write, which would add all the statements in this file to our program as if we have typed them in.

It is common for the files that are to be included to have a .h extension. This extension stands for 'header file', possibly because it contains statements which when included go to the head of your program. The prototypes of all the library functions are grouped into different categories and then stored in different header files. For example prototypes of all mathematics related functions are stored in the header file 'math.h', prototypes of console input/output functions are stored in the header file 'conio.h', and so on.

Actually there exist two ways to write **#include** statement. These are:

```
#include "filename"  
#include <filename>
```

The meaning of each of these forms is given below:

<pre>#include "goto.c"</pre>	This command would look for the file goto.c in the current directory as well as the specified list of directories as mentioned in the include search path that might have been set up.
------------------------------	---

<pre>#include <goto.c></pre>	This command would look for the file goto.c in the specified list of directories only.
------------------------------------	---

The include search path is nothing but a list of directories that would be searched for the file being included. Different C compilers let you set the search path in different manners. If you are using Turbo C/C++ compiler then the search path can be set up by selecting 'Directories' from the 'Options' menu. On doing this

a dialog box appears. In this dialog box against ‘Include Directories’ we can specify the search path. We can also specify multiple include paths separated by ‘;’ (semicolon) as shown below:

```
c:\tc\lib ; c:\mylib ; d:\libfiles
```

The path can contain maximum of 127 characters. Both relative and absolute paths are valid. For example ‘..\dir\incfiles’ is a valid path.

Conditional Compilation

We can, if we want, have the compiler skip over part of a source code by inserting the preprocessing commands **#ifdef** and **#endif**, which have the general form:

```
#ifdef macroname
    statement 1 ;
    statement 2 ;
    statement 3 ;
#endif
```

If **macroname** has been **#defined**, the block of code will be processed as usual; otherwise not.

Where would **#ifdef** be useful? When would you like to compile only a part of your program? In three cases:

- (a) To “comment out” obsolete lines of code. It often happens that a program is changed at the last minute to satisfy a client. This involves rewriting some part of source code to the client’s satisfaction and deleting the old code. But veteran programmers are familiar with the clients who change their mind and want the old code back again just the way it was.

Now you would definitely not like to retype the deleted code again.

One solution in such a situation is to put the old code within a pair of `/* */` combination. But we might have already written a comment in the code that we are about to “comment out”. This would mean we end up with nested comments. Obviously, this solution won’t work since we can’t nest comments in C.

Therefore the solution is to use conditional compilation as shown below.

```
main()  
{  
    #ifdef OKAY  
        statement 1 ;  
        statement 2 ; /* detects virus */  
        statement 3 ;  
        statement 4 ; /* specific to stone virus */  
    #endif  
  
    statement 5 ;  
    statement 6 ;  
    statement 7 ;  
}
```

Here, statements 1, 2, 3 and 4 would get compiled only if the macro `OKAY` has been defined, and we have purposefully omitted the definition of the macro `OKAY`. At a later date, if we want that these statements should also get compiled all that we are required to do is to delete the **`#ifdef`** and **`#endif`** statements.

- (b) A more sophisticated use of **`#ifdef`** has to do with making the programs portable, i.e. to make them work on two totally different computers. Suppose an organization has two

different types of computers and you are expected to write a program that works on both the machines. You can do so by isolating the lines of code that must be different for each machine by marking them off with **#ifdef**. For example:

```
main()  
{  
    #ifdef INTEL  
        code suitable for a Intel PC  
    #else  
        code suitable for a Motorola PC  
    #endif  
    code common to both the computers  
}
```

When you compile this program it would compile only the code suitable for a Intel PC and the common code. This is because the macro INTEL has not been defined. Note that the working of **#ifdef** - **#else** - **#endif** is similar to the ordinary **if** - **else** control instruction of C.

If you want to run your program on a Motorola PC, just add a statement at the top saying,

```
#define INTEL
```

Sometimes, instead of **#ifdef** the **#ifndef** directive is used. The **#ifndef** (which means 'if not defined') works exactly opposite to **#ifdef**. The above example if written using **#ifndef**, would look like this:

```
main()  
{  
    #ifndef INTEL  
        code suitable for a Intel PC  
    #else  
        code suitable for a Motorola PC  
}
```



```

    #endif
    code common to both the computers
}

```

- (c) Suppose a function **myfunc()** is defined in a file ‘myfile.h’ which is **#included** in a file ‘myfile1.h’. Now in your program file if you **#include** both ‘myfile.h’ and ‘myfile1.h’ the compiler flashes an error ‘Multiple declaration for **myfunc**’. This is because the same file ‘myfile.h’ gets included twice. To avoid this we can write following code in the header file.

```

/* myfile.h */
#ifndef __myfile_h
    #define __myfile_h

    myfunc( )
    {
        /* some code */
    }

#endif

```

First time the file ‘myfile.h’ gets included the preprocessor checks whether a macro called **__myfile_h** has been defined or not. If it has not been then it gets defined and the rest of the code gets included. Next time we attempt to include the same file, the inclusion is prevented since **__myfile_h** already stands defined. Note that there is nothing special about **__myfile_h**. In its place we can use any other macro as well.

#if and #elif Directives

The **#if** directive can be used to test whether an expression evaluates to a nonzero value or not. If the result of the expression is nonzero, then subsequent lines upto a **#else**, **#elif** or **#endif** are compiled, otherwise they are skipped.

A simple example of **#if** directive is shown below:

```
main()  
{  
    #if TEST <= 5  
        statement 1 ;  
        statement 2 ;  
        statement 3 ;  
    #else  
        statement 4 ;  
        statement 5 ;  
        statement 6 ;  
    #endif  
}
```

If the expression, **TEST <= 5** evaluates to true then statements 1, 2 and 3 are compiled otherwise statements 4, 5 and 6 are compiled. In place of the expression **TEST <= 5** other expressions like **(LEVEL == HIGH || LEVEL == LOW)** or **ADAPTER == CGA** can also be used.

If we so desire we can have nested conditional compilation directives. An example that uses such directives is shown below.

```
#if ADAPTER == VGA  
    code for video graphics array  
#else  
    #if ADAPTER == SVGA  
        code for super video graphics array  
    #else  
        code for extended graphics adapter  
    #endif  
#endif
```

The above program segment can be made more compact by using another conditional compilation directive called **#elif**. The same program using this directive can be rewritten as shown below.

Observe that by using the **#elif** directives the number of **#endifs** used in the program get reduced.

```
#if ADAPTER == VGA
    code for video graphics array
#elif ADAPTER == SVGA
    code for super video graphics array
#else
    code for extended graphics adapter
#endif
```

Miscellaneous Directives

There are two more preprocessor directives available, though they are not very commonly used. They are:

- (a) **#undef**
- (b) **#pragma**

#undef Directive

On some occasions it may be desirable to cause a defined name to become ‘undefined’. This can be accomplished by means of the **#undef** directive. In order to undefine a macro that has been earlier **#defined**, the directive,

```
#undef macro template
```

can be used. Thus the statement,

```
#undef PENTIUM
```

would cause the definition of **PENTIUM** to be removed from the system. All subsequent **#ifdef PENTIUM** statements would evaluate to false. In practice seldom are you required to undefine a macro, but for some reason if you are required to, then you know that there is something to fall back upon.

#pragma Directive

This directive is another special-purpose directive that you can use to turn on or off certain features. Pragas vary from one compiler to another. There are certain pragmas available with Microsoft C compiler that deal with formatting source listings and placing comments in the object file generated by the compiler. Turbo C/C++ compiler has got a pragma that allows you to suppress warnings generated by the compiler. Some of these pragmas are discussed below.

- (a) **#pragma startup** and **#pragma exit**: These directives allow us to specify functions that are called upon program startup (before **main()**) or program exit (just before the program terminates). Their usage is as follows:

```
void fun1( ) ;
void fun2( ) ;

#pragma startup fun1
#pragma exit fun2

main( )
{
    printf ( "\nInside main" ) ;
}

void fun1( )
{
    printf ( "\nInside fun1" ) ;
}

void fun2( )
{
    printf ( "\nInside fun2" ) ;
}
```